Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Модели данных и системы управления базами данных»

ОТЧЕТ
к лабораторной работе №1
на тему:
«ПРОСТЫЕ ПРОЦЕДУРЫ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ИОВИЧ Евгения Александровна

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Создайте таблицу MyTable (id number, val number).

2 Напишите анонимный блок, который записывает в таблицу MyTable 10 000 целых случайных записей.

3 Напишите собственную функцию, которая выводит TRUE если четных значений val в таблице MyTable больше, FALSE если больше нечетных значений и EQUAL, если количество четных и нечетных равно.

4 Напишите функцию, которая по введенному значению ID, сгенерирует и выведет в консоль текстовое значение команды insert для вставки указанной строки.

5 Написать процедуры, реализующие DML операции (INSERT, UPDATE, DELETE) для указанной таблицы.

6 Создайте функцию, вычисляющую общее вознаграждение за год. На вход функции подаются значение месячной зарплаты и процент годовых премиальных. В общем случае общее вознаграждение = (1 + процент годовых премиальных) * 12 * значение месячной зарплаты. При этом предусмотреть что процент вводится как целое число, и требуется преобразовать его к дробному. Предусмотреть защиту от ввода некорректных данных.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

PL/SQL (Procedural Language/Structured Query Language) – это процедурное расширение языка SQL, разработанное компанией Oracle для работы с базами данных Oracle. PL/SQL интегрирован в сервер базы данных, что обеспечивает высокую производительность при обработке данных. Основной единицей PL/SQL является блок, который может быть анонимным или именованным (процедура, функция, пакет, триггер).

Переменные и константы объявляются в разделе DECLARE с указанием имени, типа и, при необходимости, начального значения. PL/SQL поддерживает стандартные типы данных SQL (NUMBER, VARCHAR2, DATE и др.), а также специфические типы, такие как BOOLEAN, BINARY_INTEGER.

Процедуры и функции являются именованными блоками PL/SQL, хранимыми на сервере базы данных. Процедура выполняет определенные действия и может иметь входные (IN), выходные (OUT) и входно-выходные (IN OUT) параметры. Функция возвращает значение и обязательно содержит оператор RETURN в исполняемой части.

DML-операции (INSERT, UPDATE, DELETE) могут выполняться непосредственно в PL/SQL-блоках. Для работы с транзакциями используются операторы COMMIT и ROLLBACK. Обработка исключений осуществляется в разделе EXCEPTION с помощью конструкций WHEN.

В контексте данной лабораторной работы необходимо освоить написание анонимных блоков, создание функций и процедур для выполнения операций с данными, а также обработку исключений для обеспечения корректности работы.

3 ВЫПОЛНЕНИЕ РАБОТЫ

1 Создадим таблицу MyTable, предназначенную для хранения тестовых данных. Таблица содержит два столбца: id и val, оба числового типа. Столбец id используется в качестве идентификатора записи, а столбец val – для хранения целочисленных значений, над которыми в дальнейшем выполняются вычисления и анализ.

Листинг 3.1 – Создание таблицы MyTable

```
CREATE TABLE MyTable (
    id NUMBER PRIMARY KEY,
    val NUMBER
);
```

2 Объявим анонимный PL/SQL-блок, для заполнения таблицы 10 000 случайными целыми числами. В данном блоке применяется цикл FOR, который выполняется 10 000 раз. На каждой итерации в таблицу вставляется запись с уникальным идентификатором (от 1 до 10 000) и случайным значением val в диапазоне от 1 до 10 000, генерируемым с помощью функции DBMS_RANDOM.VALUE. Оператор TRUNC используется для получения целого числа. После выполнения всех вставок выполняется COMMIT для фиксации изменений. Результат работы анонимного блока продемонстрирован на рисунке 3.1.

	ID	VAL
9976	9977	215
9977	9978	6707
9978	9979	9193
9979	9980	4368
9980	9981	7042
9981	9982	9107
9982	9983	7379
9983	9984	3455
9984	9985	2695
9985	9986	825
9986	9987	1359
9987	9988	1943
9988	9989	8503
9989	9990	8470
9990	9991	1782
9991	9992	7290
9992	9993	7127
9993	9994	8288
9994	9995	9229
9995	9996	847
9996	9997	7728
9997	9998	5843
9998	9999	4323
9999	1	1000
10000	10000	1000

9 501-10 000 of 10 000

Рисунок 3.1 – Заполнение таблицы случайными данными

Код анонимного блока продемонстрирован в листинге 3.2.

Листинг 3.2 – Анонимный блок для заполнения таблицы

```
BEGIN
    FOR i IN 1..10000 LOOP
        INSERT INTO MyTable VALUES (i, TRUNC(DBMS_RANDOM.VALUE(1, 10000)));
    END LOOP;
    COMMIT;
END;
```

3 Следующим этапом создается функция CHECK_EVEN_ODD_RATIO, которая анализирует распределение четных и нечетных значений в столбце val таблицы MyTable. В функции объявляются две переменные v_even и v_odd типа NUMBER. С помощью двух SELECT-запросов с использованием агрегатной функции COUNT и условия MOD(val,2) подсчитывается количество четных и нечетных значений соответственно. Затем с помощью условного оператора IF-ELSIF-ELSE функция возвращает строковое значение: 'TRUE' если четных значений больше, 'FALSE' если нечетных больше, и 'EQUAL' если количества равны. Функция создается с использованием CREATE OR REPLACE FUNCTION и вызывается через SELECT для проверки результата.

Листинг 3.3 – Функция для анализа распределения чётных и нечётных чисел

```
CREATE OR REPLACE FUNCTION CHECK_EVEN_ODD_RATIO
RETURN VARCHAR2
IS
    v_even NUMBER;
    v_odd  NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_even FROM MyTable WHERE MOD(val, 2) = 0;
    SELECT COUNT(*) INTO v_odd FROM MyTable WHERE MOD(val, 2) != 0;

    IF v_even > v_odd THEN
        RETURN 'TRUE';
    ELSIF v_odd > v_even THEN
        RETURN 'FALSE';
    ELSE
        RETURN 'EQUAL';
    END IF;
END CHECK_EVEN_ODD_RATIO;
```

4 Для генерации текстовой команды INSERT по заданному идентификатору создается функция GENERATE_INSERT_COMMAND. Функция принимает один входной параметр p_id типа NUMBER. В теле функции объявляется переменная v_val для хранения значения val из таблицы и переменная v_sql типа VARCHAR2 для формирования строки команды. С помощью SELECT INTO значение val выбирается из таблицы MyTable по указанному id. Если запись с таким id существует, формируется строка вида 'INSERT INTO MyTable VALUES (id, val);'. Для вывода команды в консоль используется процедура DBMS_OUTPUT.PUT_LINE. Функция возвращает

сформированную строку. В секции EXCEPTION обрабатываются возможные ошибки: если запись с указанным id не найдена (NO_DATA_FOUND), функция возвращает сообщение об ошибке; для всех прочих ошибок возвращается сообщение с текстом ошибки из SQLERRM. Функция тестируется на нескольких значениях id, включая несуществующие и граничные.

Листинг 3.4 – Генерация текстовой команды INSERT по заданному id

```
CREATE OR REPLACE FUNCTION GENERATE_INSERT_COMMAND(p_id NUMBER)
RETURN VARCHAR2 IS
    v_val NUMBER;
    v_sql VARCHAR2(500);
BEGIN
    SELECT val INTO v_val FROM MYTABLE WHERE p_id = id;

    v_sql := 'INSERT INTO MyTable VALUES (' || p_id || ', ' || v_val || ');';
    DBMS_OUTPUT.PUT_LINE(v_sql);

    RETURN v_sql;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'ERROR! id = ' || p_id || ' not found in MyTable!';
    WHEN OTHERS THEN
        RETURN 'ERROR: ' || SQLERRM;
END GENERATE_INSERT_COMMAND;
```

Результат работы функция на различных тестовых значениях продемонстрирован на рисунке 3.2.

	INPUT_VALUE	RESULT
1	-1	ERROR! id = -1 not found in MyTable!
2	0	ERROR! id = 0 not found in MyTable!
3	1	INSERT INTO MyTable VALUES (1, 1000);
4	10000	INSERT INTO MyTable VALUES (10000, 1000);
5	10001	ERROR! id = 10001 not found in MyTable!

Рисунок 3.2 – Генерация текстовой команды INSERT

5 Согласно заданию, реализуются процедуры для операций DML: UPDATE, INSERT и DELETE:

Процедура UPDATE_MYTABLE предназначена для обновления значения val по заданному id. Процедура принимает два параметра: p_id и p_val. Вначале с помощью SELECT COUNT(*) проверяется существование записи с указанным id. Если запись существует (v_exists = 1), выполняется оператор UPDATE, после которого изменения фиксируются с помощью COMMIT, и в консоль выводится сообщение об успешном обновлении. Если запись не найдена, выводится сообщение об ошибке. Исключительные ситуации обрабатываются в секции EXCEPTION, где выводится сообщение с описанием ошибки. Смотреть рисунок 3.3 с тестами этой процедуры.

```
[2026-01-25 02:06:07] SYSTEM> CALL UPDATE_MYTABLE(-1, 1000)
[2026-01-25 02:06:07] completed in 6 ms
ERROR update: id = -1 not found!
[2026-01-25 02:06:07] SYSTEM> CALL UPDATE_MYTABLE(0, 1000)
[2026-01-25 02:06:07] completed in 2 ms
ERROR update: id = 0 not found!
[2026-01-25 02:06:07] SYSTEM> CALL UPDATE_MYTABLE(1, 1000)
[2026-01-25 02:06:07] completed in 5 ms
Successful update: id = 1 new value = 1000
[2026-01-25 02:06:07] SYSTEM> CALL UPDATE_MYTABLE(10000, 1000)
[2026-01-25 02:06:07] completed in 3 ms
Successful update: id = 10000 new value = 1000
[2026-01-25 02:06:07] SYSTEM> CALL UPDATE_MYTABLE(10001, 1000)
[2026-01-25 02:06:07] completed in 3 ms
Successful update: id = 10001 new value = 1000
```

Рисунок 3.3 – Процедура UPDATE_MYTABLE

Процедура INSERT_INTO_MYTABLE выполняет вставку новой записи в таблицу MyTable. Процедура принимает параметры p_id и p_val. Для обеспечения корректности данных добавлена проверка: если p_id меньше 1, с помощью RAISE_APPLICATION_ERROR вызывается пользовательское исключение с кодом ошибки -20001 и соответствующим сообщением. Если проверка пройдена, выполняется оператор INSERT и COMMIT, после чего выводится сообщение об успешной вставке. В секции EXCEPTION обрабатывается исключение DUP_VAL_ON_INDEX (попытка вставки дубликата значения первичного ключа) и все прочие исключения с выводом сообщения об ошибке. Смотреть рисунок 3.4 с тестами этой процедуры.

```
[2026-01-25 02:05:02] SYSTEM> CALL INSERT_INTO_MYTABLE(-1, 1000)
[2026-01-25 02:05:02] completed in 5 ms
ERROR: ORA-20001: p_id = -1 must be > 0
[2026-01-25 02:05:02] SYSTEM> CALL INSERT_INTO_MYTABLE(0, 1000)
[2026-01-25 02:05:02] completed in 3 ms
ERROR: ORA-20001: p_id = 0 must be > 0
[2026-01-25 02:05:02] SYSTEM> CALL INSERT_INTO_MYTABLE(1, 1000)
[2026-01-25 02:05:02] completed in 7 ms
ERROR insert: row with id = 1 already exist!
[2026-01-25 02:05:02] SYSTEM> CALL INSERT_INTO_MYTABLE(10000, 1000)
[2026-01-25 02:05:02] completed in 3 ms
ERROR insert: row with id = 10000 already exist!
[2026-01-25 02:05:02] SYSTEM> CALL INSERT_INTO_MYTABLE(10001, 1000)
[2026-01-25 02:05:02] completed in 4 ms
Successful insert: id = 10001, val = 1000
```

Рисунок 3.4 – Процедура INSERT_INTO_MYTABLE

Процедура DELETE_FROM_MYTABLE удаляет запись из таблицы MyTable по заданному идентификатору p_id. Логика аналогична процедуре UPDATE: сначала проверяется существование записи. Если запись найдена, выполняется DELETE и COMMIT, выводится сообщение об успешном удалении. Если запись не существует, выводится сообщение об ошибке. Также предусмотрена обработка исключений в секции EXCEPTION. Смотреть рисунок 3.5 с тестами этой процедуры.

```
[2026-01-25 02:05:27] SYSTEM> CALL DELETE_FROM_MYTABLE(-1)
[2026-01-25 02:05:27] completed in 8 ms
ERROR delete: id = -1 not found!
[2026-01-25 02:05:27] SYSTEM> CALL DELETE_FROM_MYTABLE(0)
[2026-01-25 02:05:27] completed in 3 ms
ERROR delete: id = 0 not found!
[2026-01-25 02:05:27] SYSTEM> CALL DELETE_FROM_MYTABLE(1)
[2026-01-25 02:05:27] completed in 5 ms
Successful delete: id = 1
[2026-01-25 02:05:27] SYSTEM> CALL DELETE_FROM_MYTABLE(10000)
[2026-01-25 02:05:27] completed in 3 ms
Successful delete: id = 10000
[2026-01-25 02:05:27] SYSTEM> CALL DELETE_FROM_MYTABLE(10001)
[2026-01-25 02:05:27] completed in 3 ms
Successful delete: id = 10001
```

Рисунок 3.5 – Процедура DELETE_FROM_MYTABLE

6 Функция CALCULATE_ANNUAL_REWARD для расчета общего годового вознаграждения сотрудника. Функция принимает два параметра: p_salary (месячная зарплата) и p_bonus_percent (процент годовых премиальных, целое число). В соответствии с условием, общее вознаграждение рассчитывается по формуле: (1 + p_bonus_percent/100) * 12 * p_salary. В функции реализована защита от ввода некорректных данных: проверка на NULL значений параметров, проверка на отрицательные значения зарплаты и процента премиальных. Для каждой возможной ошибки определен уникальный код пользовательского исключения (от -20001 до -20004). При обнаружении ошибки вызывается RAISE_APPLICATION_ERROR с соответствующим кодом и текстом сообщения. Если все проверки пройдены, вычисляется и возвращается результат. В секции EXCEPTION обрабатываются все непредвиденные ошибки, выводится сообщение об ошибке, и функция возвращает NULL. Функция протестирована на различных входных данных, включая NULL значения, отрицательные числа, нулевое значение зарплаты и корректные данные, что продемонстрировано на рисунке 3.6.

	PARAMETERS	RESULT
1	NULL, NULL	<null>
2	NULL, 5	<null>
3	1000, NULL	<null>
4	-1, 5	<null>
5	0, 5	0
6	4800, 12	64512
7	1600, 5.5	20256

Рисунок 3.6 – Тестирование работы функции

Обработка исключений выводиться в консоль, вместе с кодом ошибки, на рисунке 3.7.

```
[2026-01-25 02:07:16] 7 rows retrieved starting from 1 in 350 ms (execution: 15 ms, fetching: 335 ms)
ERROR: ORA-20001: Salary cannot be NULL
ERROR: ORA-20001: Salary cannot be NULL
ERROR: ORA-20002: Percent cannot be NULL
ERROR: ORA-20003: Salary must not be negative
```

Рисунок 3.7 – Обработка исключений функции

Код, описанной выше, функции CALCULATE_ANNUAL_REWARD представлен в листинге 3.5.

Листинг 3.5 – Расчёт ежегодного вознаграждения

```
CREATE OR REPLACE FUNCTION CALCULATE_ANNUAL_REWARD(p_salary NUMBER,
p_bonus_percent NUMBER)
RETURN NUMBER IS
    v_error_null_salary CONSTANT NUMBER := -20001;
    v_error_null_percent CONSTANT NUMBER := -20002;
    v_error_negative_salary CONSTANT NUMBER := -20003;
    v_error_negative_percent CONSTANT NUMBER := -20004;

    v_total_reward NUMBER;
BEGIN
    IF p_salary IS NULL THEN
        RAISE_APPLICATION_ERROR(v_error_null_salary, 'Salary cannot be NULL');
    END IF;

    IF p_bonus_percent IS NULL THEN
        RAISE_APPLICATION_ERROR(v_error_null_percent, 'Percent cannot be
NULL');
    END IF;

    IF p_salary < 0 THEN
        RAISE_APPLICATION_ERROR(v_error_negative_salary, 'Salary must not be
negative');
    END IF;

    IF p_bonus_percent < 0 THEN
        RAISE_APPLICATION_ERROR(v_error_negative_percent, 'The bonus
percentage cannot be negative');
    END IF;

    v_total_reward := (1 + p_bonus_percent / 100) * 12 * p_salary;
    RETURN v_total_reward;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
        RETURN null;
END CALCULATE_ANNUAL_REWARD;
```

ВЫВОД

В ходе выполнения лабораторной работы были успешно приобретены и закреплены практические навыки работы с языком PL/SQL в среде Oracle Database.

В рамках работы была создана тестовая таблица MyTable и с помощью анонимного PL/SQL-блока выполнено её заполнение 10 000 случайных записей. Была, также, разработана пользовательская функция CHECK_EVEN_ODD_RATIO, которая анализирует данные в таблице и возвращает результат сравнения количества четных и нечетных значений. Это позволило освоить создание функций, использование условных операторов и агрегатных SQL-запросов в PL/SQL. Создание функции GENERATE_INSERT_COMMAND для генерации DML-команд по идентификатору записи показало умение работать с динамическим формированием строк, обработкой исключительных ситуаций (NO_DATA_FOUND) и использованием пакета DBMS_OUTPUT для вывода информации. Реализация процедур INSERT_INTO_MYTABLE, UPDATE_MYTABLE, DELETE_FROM_MYTABLE и функции CALCULATE_ANNUAL_REWARD для расчета годового вознаграждения отработала навыки создания защищенного кода с валидацией входных данных, преобразованием типов и обработкой потенциально некорректных значений.

Таким образом, в ходе лабораторной работы были изучены ключевые элементы языка PL/SQL: структура блоков (анонимных и именованных), объявление переменных и констант, использование основных типов данных, создание процедур и функций, выполнение DML-операций, управление транзакциями и обработка исключений. Полученные знания формируют основу для разработки хранимых процедур и функций на стороне сервера базы данных Oracle.